Privacy Attack via Temporal Embedding Inversion

Kaicheng Xue

ybg4wb@virginia.edu

1 Introduction

Embeddings are vector representations of natural language which encode semantic information and are commonly used in different systems for pre-processing texts. Typically, the construction of embeddings is achieved through black-box function learning to obtain compressed vectors. For a long time, researchers generally believe that embeddings are complex and irreversible. The original texts cannot be reconstructed successfully. This leads to an implicit assumption of information security in many applications.

However, with increasing research devoted to large models and their internal feature inversion, it has been found that embeddings can be reconstructed under certain conditions. The study of inversion began when researchers attempted to infer training data from model outputs. It later expanded to the investigation of intermediate representations in hidden layers, and eventually extended to embeddings in natural language processing. Li et al. (2023) presented a study about embedding inversion and texts reconstruction [4]. Their experimental results indicate that embeddings contain much more information than simple compressed representations, including word orders and token-level statistical cues. This line of work demonstrates that intermediate representations can still encode recoverable information about original inputs.

The above discussion reveals a special case. It was previously assumed that using embeddings was secure for systems, but now the possibility of embeddings being vulnerable to attacks should also be considered. In particular, systems that frequently transmit and store embeddings may face substantial security risks. Retrieval-augmented generation (RAG) is a representative example. It was first introduced by Lewis et al. (2020) and deployed in applications to improve interactions between the LLM models and users within specific domains [3]. When users interact with a RAG system, the conversation history or retrieved context may be stored as embeddings in cache, web session or temporary memory buffers. And such interaction data can be intercepted, which may cause embedding leakage and poses potential privacy risks if personal information is reconstructed from these embeddings via inversion methods.

Despite growing interest in embedding inversion, most existing studies focus on single sentence inversion, which ignores information from the perspective of temporal structure, particularly in RAG chat histories. This oversight is crucial in current privacy risk assessments, as the combination of information from chat history may increase the effectiveness of embedding inversion attacks. To address this gap, we propose a temporal-aware inversion framework that incorporates conversation embeddings in history into the reconstruction process. Moreover, we explore an enhanced variant of the proposed method by replacing key components with neural network modules, aiming to take advantage of non-linear structure to achieve better performance.

Interestingly, we find that this variant does not improve performance. Through further analysis, we identify that the additional assumptions for the enhanced design do not hold. Once these assumptions are removed, this new model may fall back to a form which is similar to the proposed temporal method. The analysis provides insight about which modeling assumptions are necessary for effective inversion attacks in RAG systems.

Overall, this work introduces a temporal-aware inversion method for RAG systems, suggests the impact of temporal structure on reconstruction performance, and provides a comprehensive analysis of the assumptions for inversion modeling, revealing their limitations in realistic settings.

2 Related Work

Many studies have investigated text embedding inversion and proposed different methods. These approaches can generally be categorized into three types. The first approach is to treat the entire embedding inversion problem as a task similar to an encoder-decoder framework. Morris et al. (2023) propose a framework that receives embeddings as input and uses a decoder to generate the original text [5]. This framework reverses the input and output in a traditional encoder-decoder task, which guarantees its success at both the theoretical and empirical levels. However, this method requires a large amount of paris of original texts and embeddings for training, which may not be applicable in reality as embedding leakage is insufficient in most cases.

The second approach is to learn a mapping between the target embedding space and the attacker embedding space. Chen et al. (2025) propose a method that optimizes alignment between the victim embedding space and attacker embedding space, which reconstructs the original text by self-aligned embedding space [1]. The core idea of this framework is to minimize least square distance between two embedding spaces. This problem has closed form mathematical solution for simple computation, which largely reduces the number of

original texts needed. Nonetheless, this framework requires two embedding spaces have structural similarity, which may not be effective for models developed from different structures or training processes.

The third approach is to iteratively predict possible words given prior texts and search the best one as final result. Kim et al. (2026) propose an algorithm that iteratively produces possible candidates and projects them into the target space for ranking and selection [2]. Although this algorithm adopts similar optimization in aligning two embedding spaces as second approach, it incorporates online updates for better alignment and does not require access to the original text. This method enhances privacy attacks in the absence of the original text. Nevertheless, this algorithm relies on a linear mapping between two different embedding spaces, which may not be accurate as compared to non-linear alignment.

In summary, three methods adopt different ideas of reconstruction under various scenarios, utilizing existing mathematical results to simplify the problem and reduce computation complexity. Nonetheless, even the latest method does not consider temporal structure of dialogues in implementation, which is an unexplored aspect in attacks on RAG systems. This motivates our work to extend recent research further with a temporal architecture, aiming to fill the gap in embedding inversion attacks on RAG systems.

3 Methodology

In this section, we first define the task of conversation embedding inversion for RAG system. We then introduce the baseline algorithm and describe two architecture variants, followed by analysis of their failure modes and correction strategies.

3.1 Problem Definition

The scenario of the task is that conversation embedding between a RAG assistant and a user is leaked. The embeddings are formed in dialogue structure, and each turn is a time step. We assume that the adversary does not have access to the original texts of conversation, but can query a victim embedding model to obtain victim embeddings of arbitrary texts without accessing its internal architecture. This assumption is motivated by the widespread use of commercial embedding models in real RAG systems, many of which have open-source versions. This enables black-box access to vicitm embedding space via APIs. The objective of the task is to reconstruct the original texts in a dialogue structure from pure embeddings and extract sensitive information contained in the contents.

3.2 Base Algorithm

We adopt the embedding inversion approach proposed by Kim et al. as baseline [2]. The core idea of algorithm is as follows:

- **Next Token Prediction**: Use pre-trained LLM to predict next possible tokens based on prior text
- **Verification and Ranking**: Obtain local embedding of candidate sentences and project them into the target space by linear mapping W. Rank projected candidates by similarity-based scoring function.

  Query the victim API for corresponding victim embeddings of top-K projected candidates; Use them to update linear mapping W and re-rank.
- **Iterative Generation**: Select final top-K candidates as the output. Repeat above until maximum length is reached or stop token appears.

In general, the base algorithm performs inversion for an embedding in each iteration independently without considering contextual information. This ignores the global coherence among generated sentences.

3.3 Sliding-Window Algorithm

To overcome the limitation of the base algorithm mentioned above, we introduce a sliding window of history embeddings into inversion process. Specifically, the embeddings used for ranking in the base algorithm are modified. Suppose the algorithm has reconstructed texts up to time step t. Choose h of them and obtain their projected embeddings:

$$E_{\text{hist}} = [e_{t-h}, \ldots, e_t]$$

At time step $t+1$, for projected embedding of each candidate $e_{\text{pred_i}}$, the new embedding used for ranking is:

$$e_{\text{fusion}} = e_{\text{pred_i}} + \alpha \sum_{i=t-h}^{t} e_i$$

Here α is a weight decay coefficient. This operation is driven by the assumption that the embedding space behaves approximately linearly in the local region around a point, and semantic variation can be interpreted as continuous displacement in the embedding space. We expect history bias to shift the semantic meaning of candidates toward historical texts and to induce conversational structure.

3.4 Temporal Context Fusion Network

To enhance the sliding-window algorithm, we consider upgrading linear mapping and history embedding addition. Linear mapping is replaced by an MLP network, which has stronger approximation and expressive power for projection. History embedding addition is replaced by temporal encoder. The temporal encoder summarizes a sequence of historical embeddings by a non-linear network. And fusion module is

introduced to fuse all the information including logits of next tokens, embedding projection and history information. The backbone of this enhanced model is a non-linear structure.

We only consider training projection MLP network and fusion module. Due to their different designs, we construct loss function separately. For projection MLP network, we consider embedding consistency loss, which aligns projected embeddings $\text{MLP}(e_i)$ with the target embedding space $\tilde{e}_i$:

$$L_{\text{map}} = \sum_i \|\text{MLP}(e_i) - \tilde{e}_i\|^2$$

For fusion module, we consider reconstruction loss, which aligns fusion result with target embeddings:

$$L_{\text{rec}} = \sum \left\| r_{\text{fusion}} - r_{\text{target}} \right\|^2$$

Here r_{target} comes from projection of victim embeddings, which is a mathematical technique to convert victim embedding so that it can be compared with fusion result.

This new model is a dual-stream architecture. The flow of MLP projection generates projected embeddings of candidates from LLM next token predictions. The flow of temporal encoder produces temporal encoding of history generated texts. Fusion module fuses these two with projection of victim embedding, and similarity-based score function ranks all candidates to output final generation. The pipeline is summarized in Figure 1 with an overview of flow of projection, flow of temporal encoder, fusion module, score ranking and final generation.

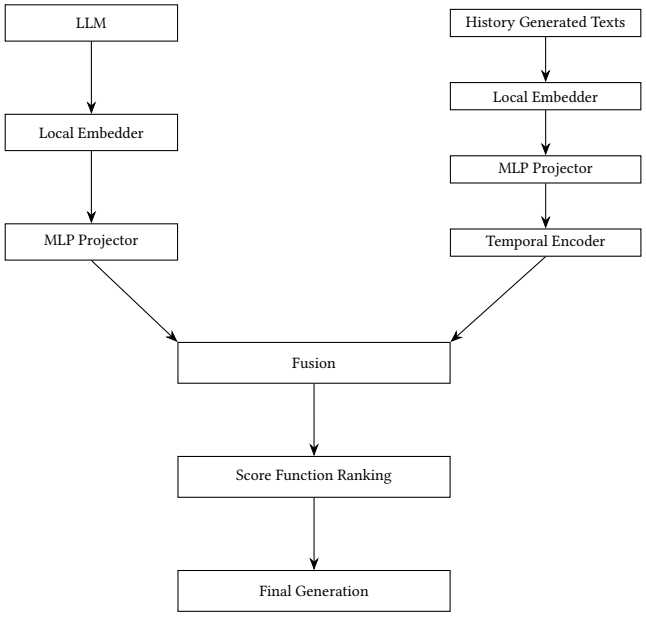


Figure 1. Proposed Temporal Context Fusion Network

3.5 Analysis of Failure Modes

For sliding-window algorithm , we observe that it does not improve quality of reconstruction texts as compared with the base algorithm. The results contain less sensitive information than expected. This behavior can be attributed to the embedding model used for converting local embeddings. This local embedder fails to generate distinguishable embedding vectors for words in texts.

For Temporal Context Fusion Network, we observe that this new architecture degrades reconstruction quality as compared to the base and sliding-window algorithm. And this can be traced to MLP projector, which fails to align two embedding spaces and convert embeddings accurately for following fusion and ranking step.

3.6 Correction Insight

For sliding-window algorithm, we identify the issue related to local embedder. And mitigation strategy is to fine tune this embedding model to adapt to the topic of our datasets.

In contrast, for Temporal Context Fusion Network, its failure originates from assumption mismatch rather than an implementation-level issue. Specifically, the assumption that a nonlinear MLP projector has greater expressive power does not hold in this case. When this assumption is corrected, the projector may fall back to a linear mapping, making it functionally equivalent to the sliding-window algorithm. As a result, we do not introduce a separate correction mechanism for Temporal Context Fusion Network, and a detailed analysis is presented in Section 6.2.

4 Dataset

According to the problem formulation, only embeddings of conversations between user and RAG system are leaked. Due to the lack of existing datasets satisfying this setting, we construct a dataset by ourselves. There are some existing Q&A datasets (e.g. SQuAD [6]) consisting of pairs of question and answer. A reasonable way is to use pre-trained LLM to rewrite each pair as multi-turn conversation to simulate dialogue between assistant and user. Also, one of the three types of sensitive information is inserted into the generated dialogue for privacy attacks, which are: (1) user profile, including name, profession and city; (2) user financial information, including bill, income and investment and (3) user health conditions, including symptom, examine result and solution.

We asked pre-trained LLM to generate 1000 combinations for each type of sensitive information and there are 3000 in total. For “user profile”, we select 1000 pair of question and answer from existing Q&A dataset, rewrite each as a conversation, and insert sensitive information of "user profile" in each of them. For “user financial information” and “user health condition”, as most Q&A datasets lack sufficient coverage of these

topics, we ask LLM to generate 1000 simulated dialogues for each type with inserted sensitive information. Finally, all sentences in conversation are converted into embedding vectors. We obtain a dataset containing 3000 conversations and corresponding embeddings as leakage.

Below are examples of three types of sensitive information:

- **User Profile**:
  Name (e.g. James)
  Profession (e.g. Surgeon)
  City (e.g. Chicago)
- **User Financial Info**:
  Bill Amount (e.g. 250)
  Income (e.g. 78500)
  Investment Return Percent (e.g. 5.2)
- **User Medical Info**:
  Symptom (e.g. Cough)
  Examine Result (e.g. Lung infection)
  Solution (e.g. Honey)

For the "user financial information", we focus on reconstructing key numbers as numerical values are also common targets in conversations.

5 Experiments

In this section, we present experiment settings, evaluation metrics and main results of all algorithms.

5.1 Experimental Setup

Models. We use Qwen3-0.6B, a model from the Qwen3 family [10], as a pre-trained LLM to predict next tokens given the preceding text. For local embedder used for converting texts into local embeddings, we use all-MiniLM-L6-v2 model [9]. For victim embedder, we use e5-base-v2 model to simulate leaked embeddings [8].

Evaluation Metrics. Two forms of metrics are used to assess outputs. For quality of reconstructed texts, we employ BLEU and ROUGE scores and embedding similarity. We conduct quality evaluation on 1000 sentences reconstruction and calculate average scores. For attack success rate, we examine how much sensitive information appears in reconstructed conversations. Each conversation contains 3 sensitive texts. And we conduct evaluation on 50 groups for each type of sensitive information, and there are 150 attacks in total for each type.

5.2 Main Results

Base & Sliding-Window Algorithm. From Table 1, it is observed that the base algorithm performs best on reconstructing texts for user medical information dataset. In comparison, the sliding-window algorithm achieves slightly higher scores on BLEU-1, BLEU-2, ROUGE-1, ROUGE-L and similarity for both user profile and user financial information, while achieving relatively same performance on the medical information dataset. Table 2 shows similar comparison in terms of attack success count. Below are some examples of reconstructed texts by the base and sliding-window algorithms.

Table 1. Quality Comparison across Different Algorithms

Category	Metric	Baseline	SW	SW (Fine-tuned)
User Profile	BLEU-1	7.13	9.0	8.43
	BLEU-2	2.39	3.14	2.79
	ROUGE-1	8.29	9.06	8.43
	ROUGE-L	6.87	7.48	6.88
	Similarity	0.8236	0.8264	0.83
Financial Info	BLEU-1	9.72	11.45	11.4
	BLEU-2	3.65	4.14	4.34
	ROUGE-1	12.62	12.98	13.22
	ROUGE-L	10.45	10.81	10.7
	Similarity	0.8656	0.8681	0.871
Medical Info	BLEU-1	11.44	12.31	12.32
	BLEU-2	4.51	4.61	4.8
	ROUGE-1	12.72	12.37	12.39
	ROUGE-L	11.06	10.06	10.67
	Similarity	0.8502	0.8468	0.8525

Table 2. Attack Success Count Comparison (Total=150)

Category	Attributes	Baseline	SW	SW (Fine-tuned)
User Profile	name	8	14	23
	profession	7	11	20
	city	8	4	18
Financial Info	bill	6	6	8
	income	1	0	1
	investment	0	2	3
Medical Info	symptom	44	45	48
	result	16	14	16
	solution	12	12	15

User Profile Example:

- **Original Text:** Ive considered teaching or medical consulting I love sharing knowledge but Im not sure if Id still feel fulfilled
- **Base Algorithm:** Answer: (
- **Sliding-Window:** Assistant: (I would recommend considering consulting a healthcare provider or a certified medical practitioner to explore all potential

In the example above, it is found that the base algorithm cannot generate enough words. This is because that the signals in embedding of long text are average, which causes algorithm performance unstable with no texts generated in such case. As mentioned in evaluation metric, quality scores are averaged over all samples in each dataset. And those samples

which have no reconstructed texts significantly decrease average scores, which results in poor quality performance for user profile dataset as showed in Table 1. Another observation is that, in the example above, the sliding-window algorithm reconstructs the word "Assistant", which maintains the dialogue structure between the user and the assistant.

Financial Info Example:

- **Original Text:** Got it I also have a side investment that earns 5.2 annually Should I be reinvesting the returns
- **Base Algorithm:** Question : 5-year investment with annual return 2.45% and I want to
- **Sliding-Window:** Question : 5-year investment returns. Suppose I have a $5 $-year investment

In the example above, both base and sliding-window algorithms generate "5", "investment" and "return" in reconstructed texts. However, embedding models usually capture semantic meanings first rather than numerical values. Numbers may not be strong signals in embedding representations, which makes them difficult for algorithms to reconstruct. Therefore, the attack success count for user financial information dataset is relatively low for these two algorithms.

Medical Info: Example

- **Original Text:** Ive had a persistent cough for about a week now and its getting worse
- **Base Algorithm:** Question: I am a patient with a chronic cough that has persisted for weeks.
- **Sliding-Window:** Question: I am a patient with a chronic cough with persistent symptoms. Please

In the example above, both base and sliding-window algorithm performs well. Their reconstructions are close to the original text in semantic meaning and key words, such as "cough" and "persist". Table 1 and Table 2 demonstrate that both algorithms perform best on user medical information dataset.

Sliding-Window Algorithm Fine-tuned version.
The previous section illustrates that the sliding-window algorithm provides little to no improvement in attack success count, especially on the user financial information dataset. As discussed in Section 3.6, this issue can be attributed to embedding model used for converting candidate sentences into local embeddings. With fine-tuned local embedder, as the Table 2 shows, the sliding-window algorithm achieves a much higher attack success count on user profile dataset. The Figure 2 presents overall attack success rate for three algorithms. The sliding-window algorithm with a fine-tuned embedder has best performance, achieving success rates of 40.67% and 52.67% on user profile and user medical information datasets.

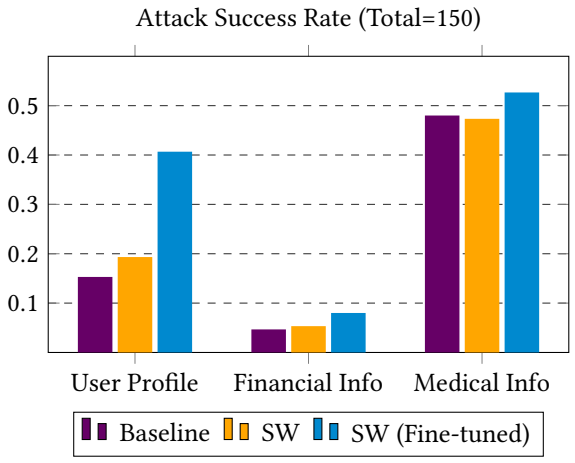


Figure 2. Attack Success Rate across Algorithms

Temporal Context Fusion Network (TCFN). This architecture is a non-linear extension of the sliding-window algorithm. However, as mentioned in Section 3.6, the assumption of using non-linear structure to improve performance fails as indicated by the training result. Thus, there are no further evaluations for this model. Here we list some examples of reconstructed texts in this model generation, which are confusing and incomprehensible.

- **Original Text:** Ive had a persistent cough for about a week now and its getting worse
- **TCFN:** Full response starts after padding import stripe -card -number pin holder screen goes dark with swipe gestures
- **Original Text:** Im sorry to hear that Have you noticed any other symptoms like fever or shortness of breath
- **TCFN:** Ad Blue Bl uen et s ... each month ! Ad Sh Hello All Human Tuesday Answer

6 Discussion

In this section, we first focus on investigating observations from sliding-window algorithm and its fine-tuned version. Then we discuss the structure of Temporal Context Fusion Network, analyze assumptions mismatch and provide comparison between it and the sliding-window algorithm.

6.1 Embedding Model Improvement

In the original sliding-window algorithm, an embedding model is used to convert candidate sentences into local embeddings, and linear mapping W projects them into victim embedding space for following scoring and ranking. From results of user profile and user financial information dataset, it can be inferred that the scoring function cannot separate correct candidates and others. Two components are responsible for expressing candidate sentences as vector representations.

One is local embedder and another is linear mapping W. In the sliding-window algorithm, it is assumed that two embedding spaces are linearly transformable, which is regarded as a linear least square problem. W can be updated well by a closed form solution with enough pairs of inputs and targets. Consequently, W updating is a relatively fixed process, which may not cause effects for following steps. This implies that the local embedder is the cause of the problem in the scoring function. Additionally, we found that the local embedding model, all-MiniLM-L6-v2, had not been trained on extensive financial corpora or daily life topics. The embeddings of texts from these two domains may not be distinguishable for scoring function to select out better prediction. The mitigation strategy is to fine tune the local embedder.

Since fine-tuning may cause damage to original embedding model performance, we only consider fine tuning one epoch on self-constructed finance corpus. In this process, we emphasize "bill", "income" and "investment" in particular. After fine-tuning local embedding model, the attack success count increases more significantly in user profile dataset than in user financial information dataset. This is an interesting observation. The human-imposed alignment in fine-tuning forces the embedding model to first differentiate texts containing three words above and ignore numerical values. Internally, the embedding model balances information importance, and other signals including those in the user profile become much stronger. This explains why fine-tuning on finance corpus improves performance on user profile dataset. This observation provides an important insight: in real-world embedding inversion attacks, where the domain of original texts is unknown and fine-tuning on specific topic is not feasible, selectively suppressing certain internal signals of embedding model may allow self-balance in it and amplify signals in other domains, which enables faster inference of information types in the original texts.

6.2 Non-linear Structure Assumption

In the Temporal Context Fusion Network, the linear mapping W is replaced with a non-linear MLP projector. This new projector is expected to align two embedding spaces more accurately. Nevertheless, the assumption that embedding spaces can be aligned by non-linear structure is not valid in this model design. Although non-linear structure has stronger approximation and expressive power, it aligns embedding space in a distorted way, which causes projector inconsistent in converting embeddings. Another reason is that the training loss function, mean squared error, enforces point-wise alignment in the embedding space, rather than structural level. A direct solution is to use linear structure for projection. Obviously, this changes the model design back to the sliding-window algorithm, which already has a quick and stable method to align two embedding spaces.

The sliding window-algorithm uses prediction logits, history information and candidates for computing scores function and ranking. All of them are combined together by addition. This is from the assumption mentioned in Secion 3.3: the geometry of the embedding space can be manipulated through small additive perturbations. In the Temporal Context Fusion Network, the addition operation is replaced with a fusion module. Prediction logits, history information and candidates are fused in multiple hidden layers. This complicates the scoring computation and signals alignment becomes unstable. Even with weighting coefficients used to control signals, the non-linear fusion module distorts all parts to a degree that makes conventional manipulation impossible. A reasonable solution is to reduce non-linear layers or remove them. Undoubtedly, this also changes the model design back to the sliding-window algorithm or a similar form. Hence, once these assumptions in the new architecture are modified or removed, the new model effectively returns to the framework of sliding-window algorithm. In contratst, sliding-window algorithm is relatively more robust.

7 Conclusion

In this work, we proposed a sliding-window algorithm and Temporal Context Fusion Network to address conversation embedding inversion for RAG system. We analyzed observations, problems and solutions from experiment results. The sliding-window algorithm with adjustment on embedder achieves best performance in attack success rate. We also compared the sliding-window algorithm with the Temporal Context Fusion Network, identifying key assumptions that can be applied to the embedding inversion task, which lay the foundation for future research.

8 Future Work

Singh (2023) proposed a method to analyze transformer dynamics as movement through embedding space [7]. This suggests a direction, namely constructing the evolution trajectory of embeddings. One way is autoregressive modeling. Given embeddings in e_{t-h} to e_t, the model learns e_{t+1} to predict the next sentence embedding. In cases of partial embedding leakage, such a predictive model, combined with the sliding-window algorithm, may help restore missing embeddings and reconstruct incomplete conversations.

9 Limitations

In this work, there are some limitations in evaluation and dataset. In evaluation, the quality metric needs to assess fluency and semantic meaning of reconstructed texts. Some reconstructed sentences contain sufficient information compared to the original text, but differ in wording or sentence structure. Unfortunately, this cannot be evaluated by BLEU or ROUGE scores. A pre-trained LLM may be used for this assessment. However, hallucinations in LLMs may lead to

unreliable evaluation results. In dataset, there are too many extremely long texts. This is rarely observed in real RAG conversations and causes instability and incoherent generation in the algorithms.

References

[1] Yiyi Chen, Qiongkai Xu, and Johannes Bjerva. 2025. Algen: Few-shot inversion attacks on textual embeddings via cross-model alignment and generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 24330–24348.

[2] Doohyun Kim, Donghwa Kang, Kyungjae Lee, Hyeongboo Baek, and Brent Byunghoon Kang. 2026. Zero2Text: Zero-Training Cross-Domain Inversion Attacks on Textual Embeddings. *arXiv preprint arXiv:2602.01757* (2026).

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[4] Haoran Li, Mingshi Xu, and Yangqiu Song. 2023. Sentence embedding leaks more information than you expect: Generative embedding inversion attack to recover the whole sentence. In *Findings of the Association for Computational Linguistics: ACL 2023*. 14022–14040.

[5] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M Rush. 2023. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 12448–12460.

[6] Pranav Rajpurkar et al. 2016. SQuAD Explorer. https://rajpurkar.github.io/SQuAD-explorer/. Accessed: 2026-03-15.

[7] Sumeet S Singh. 2023. Analyzing transformer dynamics as movement through embedding space. *arXiv preprint arXiv:2308.10874* (2023).

[8] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533* (2022).

[9] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems* 33 (2020), 5776–5788.

[10] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).